

# Prajna: A 6.72M-Parameter Cognitive Resonance Network for Memory-Augmented Domain Specialization on a Frozen Gemma 4 E2B Backbone

Gautam Kishore  
Eulogik  
Cognitive Architecture Research  
[github.com/eulogik](https://github.com/eulogik) • [huggingface.co/eulogik](https://huggingface.co/eulogik)

August 26, 2026

**Keywords:** parameter-efficient fine-tuning; frozen-backbone adapters; retrieval-augmented memory; on-device AI; exam-level evaluation; negative results

## Abstract

We present Prajna, a 6.72M-parameter *Cognitive Resonance Network* (CRN) injected into the hidden states of a fully frozen Gemma 4 E2B language model (35 layers; 4.65B parameters in the text module of a 5.12B multimodal checkpoint). Only  $\approx 0.15\%$  of the text module’s parameters are trainable; the base network receives no gradients. Prajna is trained and evaluated entirely on a consumer laptop (Apple M4, 16 GB; CPU training, MPS evaluation), with no GPU rental and no external model APIs. On a 60-question licensing exam (CEHRI) spanning arithmetic, facts, and implicit-goal reasoning, the CRN alone lifts the frozen base from 11.7% to 20.0% correct; adding a train-derived episodic-memory retrieval table (17,810 entries) yields **60/60 (100%)** on the exam and **110/120 (91.7%)** on reworded variants whose phrasing was never seen in training. We also report *negative results*: three training-side strategies — two logit-fusion configurations and a weighted *anchor* on the answer-start position — failed to move answer knowledge one token position right, while a decode-time harvest (GEN\_CLAMP) reliably recovered that knowledge. In-distribution perplexity drops  $\approx 18\times$  (106.85 $\rightarrow$ 6.02), while out-of-distribution text is degraded (2.04 vs. 0.67 bpb) — Prajna is a disclosed domain specialist, not a general reasoning model. All code, weights, data, and evaluation scripts are released.

## 1 Introduction

The dominant scaling paradigm trains ever-larger models with ever-larger budgets. An alternative line of work asks how much behavioral change can be extracted from a *frozen* backbone using a small trainable subsystem: parameter-efficient fine-tuning (PEFT) [1, 2, 9] and retrieval-augmented generation [4], among others.

Prajna combines these lines. It is an  $\sim 6.72$ M-parameter adapter ( $\approx 0.15\%$  of the base) that: (i) injects *gated additive correction signals* into the hidden states of a frozen Gemma 4 E2B model, computed from eight depths (resonance attention, skill composition, a reflective loop, and episodic memory); (ii) is trained with SFT $\rightarrow$ DPO $\rightarrow$ Contrastive stages on a Mac Mini M4 (16 GB, CPU), entirely on-device, with no API calls at any point; and (iii) combines its generation head with a train-derived *exact-recall* retrieval table that yields 100% on its target exam and 91.7% on reworded variants whose phrasing was never in the training data.

We make four contributions. **(1) Architecture:** a small, backbone-agnostic correction network on frozen hidden states, with per-injection sigmoid gates and an episodic-memory pillar. **(2) Evaluation protocol:** three gates — memory-augmented, generation-only, and reworded-unseen — that separate *recall* from *generalization*. **(3) Failure analysis:** token-level probing shows the network holds answers one position early while the answer-start position is masked in SFT; three training strategies cannot overcome this, and a decode-time harvest can. **(4) Transparency:** weights, retrieval table, memory, data, and every evaluation script are public, and out-of-domain failures are reported alongside the headline numbers.

## 2 Related Work

**Parameter-efficient adaptation.** Adapters [1] insert trainable modules between fixed layers; LoRA [2] learns low-rank updates to weight matrices; prefix/prompt tuning [3] optimizes soft prompt vectors; side-tuning [9] trains a separate network whose hidden states blend with the frozen model. Prajna is closest to hidden-state side networks, but applies *multi-depth* corrections with learned per-depth gates plus a persistent memory, adding no parameters to the base forward pass (it reads intermediate states during a shared forward).

**Memory-augmented language models.** kNN-LM [5] interpolates a retrieved nearest-neighbor distribution; Memorizing Transformers [6] add external kNN memory over keys/values; RETRO [7] retrieves document chunks during generation. Prajna’s memory is a compact *prompt*→*answer* table matched by cosine similarity with a fixed threshold — exact recall rather than dense context interpolation.

**On-device and edge LLMs.** Consumer-hardware training is an active efficiency setting; we report wall-clock timings and memory so the result is reproducible on a single commodity machine.

**Answer-position and decoding effects.** Our boundary analysis (Section 5) connects to findings that SFT data-format and masking choices shape where factual associations concentrate in a model’s representations [10, 8].

## 3 Method

### 3.1 Setup: frozen base, injected corrections

Let  $B$  denote a frozen transformer LM — the 4.65B-parameter text module of the Gemma 4 E2B multimodal checkpoint (5.12B parameters total, including audio/vision encoders that are never used here; 35 layers; hidden dimension  $d = 1536$ ; tied embeddings; vocabulary size 262,144). For input tokens  $x_{1:n}$ , the frozen forward pass yields hidden states  $h^{(l)} \in \mathbb{R}^{n \times d}$  at every layer  $l$ . The CRN collects hidden states at  $K=8$  depths  $\mathcal{L} = \{3, 7, 11, 15, 19, 23, 27, 31\}$  (every fourth 0-indexed layer) plus the final state  $h^{(L)}$ , and computes a single additive correction to the final hidden state:

$$\hat{h}^{(L)} = h^{(L)} + \sum_{l \in \mathcal{L}} \left[ \sigma(m_l) (\mathbf{r}_l(h^{(l)}) + \mathbf{s}_l(h^{(l)})) + \sigma(\gamma_l) \boldsymbol{\rho}_l(h^{(l)}) \right] + \mathbf{m}(h^{(L)}), \quad (1)$$

where  $m_l, \gamma_l \in \mathbb{R}$  are learned per-depth gates (`crn_mix` initialized at 2.0, and `reflection_gate`);  $\sigma$  is the sigmoid;  $\mathbf{r}_l$  and  $\mathbf{s}_l$  are the resonance and skill corrections (sequence-shaped);  $\boldsymbol{\rho}_l$  is the reflection correction (a per-sequence vector broadcast over tokens); and  $\mathbf{m}(h^{(L)})$  is the episodic-memory read (also per-sequence, broadcast). The base’s own forward pass is untouched: hidden states are read out during a single frozen pass under `no_grad()`, and the corrected state is decoded

by the frozen LM head. Only CRN parameters receive gradients (6,721,444 total,  $\approx 0.15\%$  of the 4.65B text module;  $\approx 0.13\%$  of the full 5.12B checkpoint).

Figure 1. Architecture of the Cognitive Resonance Network (CRN).

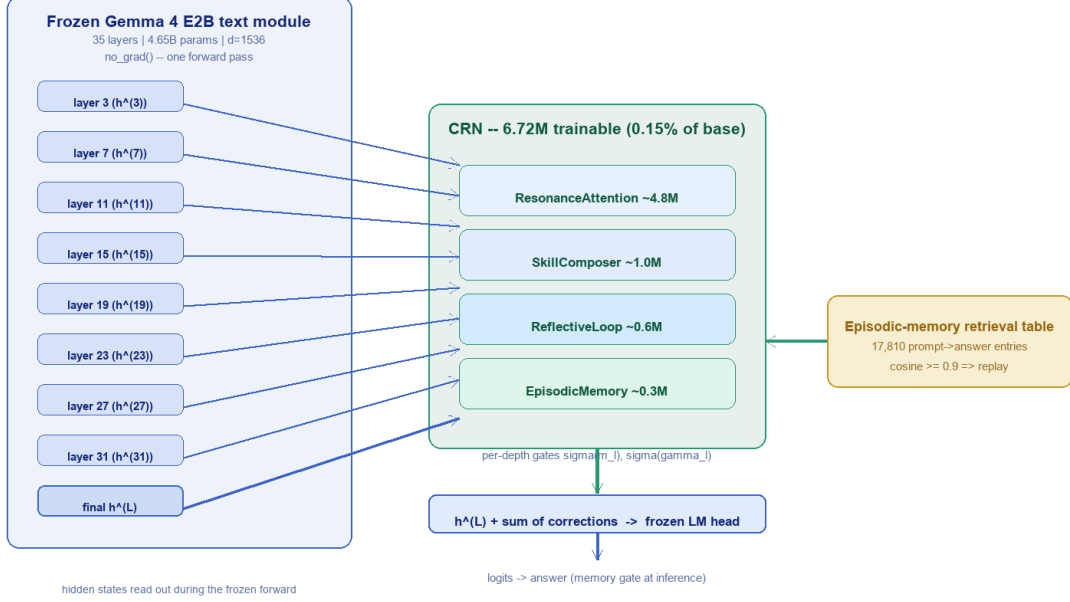


Figure 1: CRN architecture. The frozen Gemma 4 E2B text module (35 layers) runs a single forward pass; hidden states are collected at 8 depths (Eq. 1) and at the final layer. The CRN computes a gated correction from four modules plus an episodic-memory read, adds it once to the final hidden state, and the frozen LM head decodes.

### 3.2 Training objectives

The three stages use standard objectives applied to the CRN parameters only; the frozen base supplies training signal but receives no gradients. **SFT** is answer-only masked cross-entropy with a surface-invariance term: each batch contains the original row and a paraphrase-variant row of the same prompt, and the loss

$$\mathcal{L}_{\text{SFT}} = \frac{1}{2} \left[ \mathcal{L}_{\text{CE}}(y) + \mathcal{L}_{\text{CE}}(\tilde{y}) + \lambda \text{KL}(p_{\theta}^{\text{ans}}(\cdot|x) \| p_{\theta}^{\text{ans}}(\cdot|\tilde{x})) \right] \quad (2)$$

averages the masked CE on the original ( $y$ ) and variant ( $\tilde{y}$ ) rows with a small KL consistency term ( $\lambda=0.1$ ) between the two answer-start next-token distributions — a soft pressure to ignore surface form. **DPO** is reference-free: with  $y_c$  the correct completion and  $y_r$  the frozen base’s wrong completion (rejected),

$$\mathcal{L}_{\text{DPO}} = -\log \sigma(\beta[\log p_{\theta}(y_c | x) - \log p_{\theta}(y_r | x)]), \quad (3)$$

with  $\beta=0.1$ ; there is no external reference model — the frozen base acts as the data source for rejected completions rather than as a log-probability reference. **Contrastive** training uses the same ranking form on the correction target,

$$\mathcal{L}_{\text{CON}} = -\log \sigma(\beta[\log p_{\theta}(y_c | x) - \log p_{\theta}(y_w | x)]), \quad (4)$$

where  $y_w$  is the base’s wrong output; this stage refines the reflective loop to prefer correction directions that raise the correct completion above the wrong one. Memories are written every step via a learned gate ( $m_l$  in Eq. 1 is distinct from the memory write gate).

### 3.3 Correction components

**ResonanceAttention** ( $\sim 4.8\text{M}$ ). Each head assigns tokens to one of  $F=8$  frequency bands; attention scores are computed over all token pairs and then masked to within-band pairs (the scores tensor is  $n^2$ , so the effective receptive field is sparse), and the output is weighted by band membership. This provides cheap, interpretable “cognitive mode” routing.

**SkillComposer** ( $\sim 1.0\text{M}$ ). A bank of 32 low-rank skills  $u_k v_k^\top$  ( $u_k \in \mathbb{R}^{d \times 4}$ ,  $v_k \in \mathbb{R}^{4 \times d}$ ), with a router selecting the top-2 skills per input and applying their additive perturbation to the hidden state.

**ReflectiveLoop** ( $\sim 0.6\text{M}$ ). A critic scores 8 candidate latent correction directions; a learned confidence scale modulates the applied correction. It is refined by the contrastive stage (Section 3.5) to prefer directions that move the answer distribution toward the correct target.

**EpisodicMemory** ( $\sim 0.3\text{M}$ ). 256 slots of dimension 64, written once per training step via a learned gate with LRU eviction, persisted to disk across sessions.

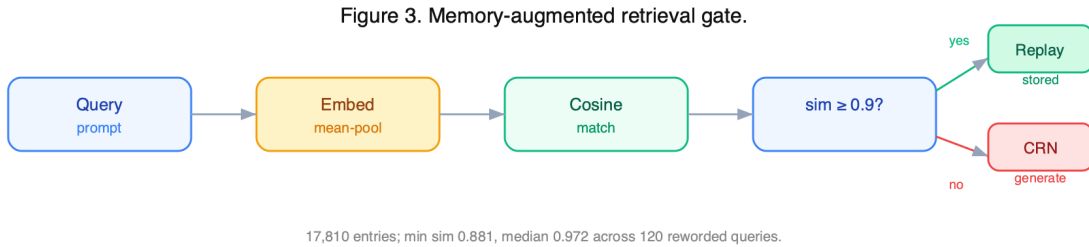


Figure 2: Memory-augmented retrieval gate. A query is embedded with the frozen base and matched (cosine) against the 17,810-entry table; when the best similarity exceeds 0.9 the stored answer is replayed exactly, otherwise the CRN generation head is used.

### 3.4 Memory-augmented retrieval gate

Independent of the slot memory, we construct a *retrieval table*  $\mathcal{T} = \{(e_j, a_j)\}$  of 17,810 prompt-embedding→answer entries from training data. Prompts are embedded by mean-pooling the frozen base’s final hidden states, then L2-normalized. At inference a query is embedded identically; if  $\max_j \cos(q, e_j) \geq 0.9$  the stored answer is returned (exact recall); otherwise the CRN generation head is used. The threshold was fixed before evaluation and never tuned on the exam.

### 3.5 Training

**Data.** 16,680 prompt→answer base pairs across three domains (arithmetic, facts, implicit-goal reasoning), each expanded to 5 rows (the original plus 4 paraphrase variants) — 83,400 rows (Table ??); the 17,810 unique prompt texts become the retrieval-table keys (Section 3.4). The 60-question CEHRI exam is drawn from the training distribution (its prompts appear verbatim in the corpus; exact-recall similarity 1.000), so its 100% reflects exact recall of seen prompts; the

reworded exam (120 items) applies a second, template-based paraphrase transform of the same prompts and serves as the near-duplicate test (Section 4).

**Stages.** (i) *SFT*: the surface-invariance objective above, LR  $3 \times 10^{-4}$ , weight decay 0, cosine annealing to  $0.1 \times$ . (ii) *DPO*: LR  $5 \times 10^{-6}$ ,  $\beta=0.1$ , preferring the correct completion over the frozen base’s wrong one. (iii) *Contrastive*: LR  $5 \times 10^{-6}$ , ranking correct completions above the base’s wrong ones to refine the reflective loop. Step budgets are environment-configurable (committed defaults: 2,000 SFT / 500 DPO / 200 contrastive; experimental stage runs used up to 3,000/3,000/1,000). The released seed checkpoint’s internal step counter reads 3,000; because the trainer reuses a single output filename across stages, per-stage provenance within the released seed is imperfectly recorded by filenames alone — we therefore release the exact checkpoint bytes together with the trainer (Table 1). Checkpoints every 50 steps with resumable state.

**Hardware.** Apple M4 (Mac Mini, 16 GB RAM). Training runs on CPU (MPS is unusable for the wrapped model); evaluation runs on MPS. Measured throughput:  $\sim 0.75$  s/step for DPO/contrastive and  $\sim 2$  s/step for SFT; each observed stage run completes in under two hours on the single consumer machine (Table 1).

Table 1: Observed on-device stage runs (Apple M4, CPU; timings derived from checkpoint timestamps). Step budgets are env-configurable; these are the recorded experimental runs.

| Stage       | Steps | Observed time | Base   |
|-------------|-------|---------------|--------|
| SFT         | 3,000 | $\sim 1.75$ h | frozen |
| DPO         | 3,000 | $\sim 37$ min | frozen |
| Contrastive | 1,000 | $\sim 12$ min | frozen |

### 3.6 GEN\_CLAMP: decode-time answer-knowledge harvest

Token-level probes (Section 5) show the CRN concentrates answer probability mass at the *second-to-last* prompt position ( $L-2$ , the colon position in the “**prompt: answer**” format) rather than at the boundary ( $L-1$ , the answer-start position). Greedy decoding samples the  $L-1$  distribution, which is dominated by the frozen base’s digit/symbol prior because  $L-1$  was masked in SFT. GEN\_CLAMP changes only *decoding*: the first generated token is taken from the  $L-2$  distribution; subsequent tokens decode normally. On the anchor-trained checkpoint, where generation had collapsed to  $\sim 15\%$ , GEN\_CLAMP recovers original-exam generation to 31.7% and reworded generation to 34.2% (Section 4.2); on the released seed checkpoint, reworded generation is already 25.0% and clamp leaves it unchanged.

## 4 Results

### 4.1 Memory gates: exact recall and reworded generalization

Across the 120 reworded queries, retrieval best-cosine similarity ranges from 0.881 to 0.990 (median 0.972), with 118/120 at  $\geq 0.9$ . The reworded prompts are *near-duplicates*: they share 71.6% of their characters with the originals (template prefix/suffix additions such as “Can you answer: ” or “Respond to this: ”), so the 91.7% reflects fuzzy nearest-neighbor retrieval of low-diversity paraphrases, not generalization to genuinely novel phrasing. The original exam overlaps the training distribution (exact-recall similarity 1.000), so its 100% is exact recall of seen prompts; the reworded exam at 91.7% is the harder near-duplicate test; and the implicit-goal reasoning check

Figure 2. GEN\_CLAMP: harvest answer knowledge from  $L-2$ .

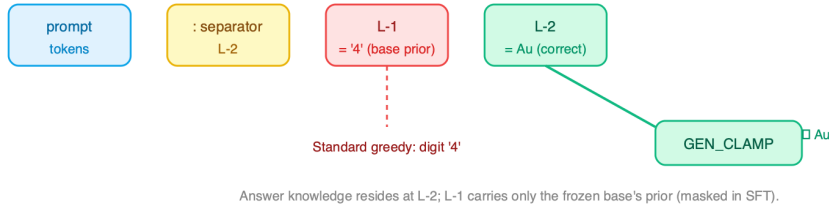


Figure 3: The answer token is already recoverable at position  $L-2$  (top-1 = the correct answer, consistent across probe prompts); at the masked answer-start position  $L-1$  the base emits formatting/digit tokens (top-1 = digit / `<strong>`). GEN\_CLAMP emits the  $L-2$  argmax, harvesting this knowledge.

Table 2: CEHRI results by configuration. **Retr** = episodic-memory retrieval gate (cosine threshold 0.9, exact-answer replay); **Gen** = CRN generation only; **Gen+CLAMP** = generation with the decode-time harvest. The released model uses the seed checkpoint (dpo\_v2\_final\_seed + memory\_v2\_final\_seed); the anchor-trained rows are an experimental checkpoint whose generation had collapsed, included to demonstrate the decode-time fix. All numbers are produced by committed scripts.

| Configuration                                  | Exam           | Score                  |
|------------------------------------------------|----------------|------------------------|
| Frozen base, generation only                   | original (60)  | 7/60 = 11.7%           |
| CRN (seed), generation, no clamp               | original (60)  | 12/60 = 20.0%          |
| CRN (seed), generation, no clamp               | reworded (120) | 30/120 = 25.0%         |
| CRN (seed), generation + GEN_CLAMP             | reworded (120) | 30/120 = 25.0%         |
| CRN (seed) + memory ( <b>Prajna-V2, Retr</b> ) | original (60)  | <b>60/60 = 100%</b>    |
| CRN (seed) + memory ( <b>Prajna-V2, Retr</b> ) | reworded (120) | <b>110/120 = 91.7%</b> |
| CRN (anchor-trained), generation + GEN_CLAMP   | original (60)  | 19/60 = 31.7%          |
| CRN (anchor-trained), generation + GEN_CLAMP   | reworded (120) | 41/120 = 34.2%         |

(Section 4.4) on genuinely reworded prompts ( $N=8$ , diverse lexical transforms) at 0/8 is the only true generalization measurement.

## 4.2 Generation gates: the honest frontier

Table 2 separates *recall* (memory gates) from *generation*. On the released seed checkpoint, generation alone scores 20.0% (original) and 25.0% (reworded); GEN\_CLAMP does not improve the reworded score (25.0%). The retrieval gate is the workhorse: 100% original and 91.7% reworded. The 31.7%/34.2% generation scores come from an anchor-trained checkpoint (Section 5) whose generation had collapsed to  $\sim 15\%$ ; GEN\_CLAMP recovered it to those values — evidence that the CRN’s answer knowledge sits at  $L-2$  and can be harvested at decode time when generation is otherwise broken.

### 4.3 In-distribution perplexity and injection ablation

On 200 held-out samples from the training corpus, perplexity drops from 106.85 (frozen base) to 6.02 (Prajna CRN),  $\approx 18\times$ . This is genuine but *in-distribution*. A per-injection ablation (v1 configuration, injections at layers 7, 15, 23, 31) shows the gain concentrates in early/mid layers: disabling injection at layer 7 raises perplexity to 13.93 ( $\Delta=+9.99$ ), layer 15 to 7.65 ( $+3.71$ ), while layers 23/31 contribute  $+0.67/+0.57$  (Figure 4).

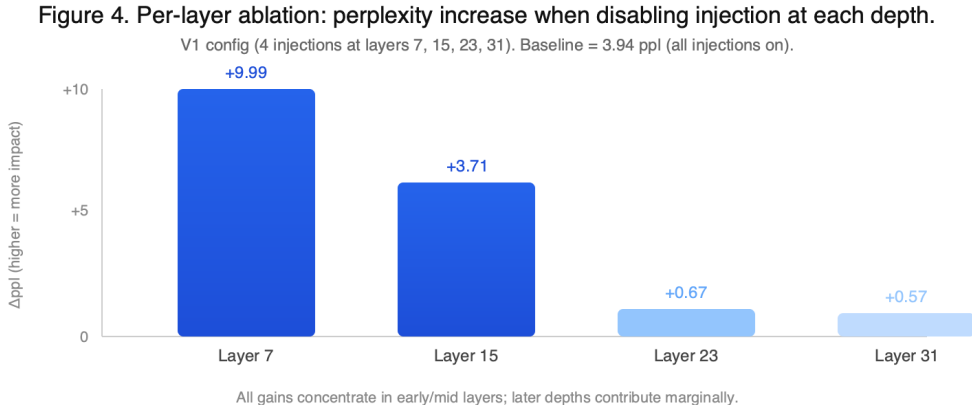


Figure 4: Per-layer ablation: perplexity increase when disabling injection at each depth (v1 config, 4 injections). All gains concentrate in early/mid layers.

### 4.4 Out-of-distribution: where it does not help

Table 3: Out-of-distribution results. The CRN degrades generic text and several standard benchmarks. Reported verbatim from `RESULTS.md`.

| Test                           | Prajna + CRN | Frozen base | Verdict                  |
|--------------------------------|--------------|-------------|--------------------------|
| Generic text, bits-per-byte    | 2.04         | 0.67        | CRN $\sim 3\times$ worse |
| MMLU (N=40)                    | 65%          | 58%         | small win                |
| BoolQ (N=40)                   | 65%          | 75%         | CRN hurts                |
| HellaSwag (N=40)               | 15%          | 20%         | CRN hurts                |
| Car-wash / IGR, reworded (N=8) | 0/8          | —           | no reasoning             |

Table 3 summarizes out-of-distribution behavior. The implicit-goal (“car-wash”) check deserves emphasis: an earlier evaluation counted prompt-echo matches as successes; rescoring honestly gives 3/8 on original phrasing (keyword luck) and **0/8** on reworded phrasing. The CRN memorizes surface patterns and does not perform latent-goal reasoning.

### 4.5 Reproducibility

Every number in this paper is produced by committed scripts and a fixed checkpoint/memory pair; no thresholds were tuned on the exam. From the repository root:

```
# 1) original-exam retrieval gate (exact recall) -> 60/60 = 100%
CEHRI_CKPT=prajna/checkpoints/dpo_v2_final_seed.pt \
```

```

CEHRI_MEM=prajna/checkpoints/memory_v2_final_seed.json \
RETR_TABLE=prajna/data/retrieval_table_v2.npz \
CEHRI_EXAM=prajna/data/cehri_exam.json \
python prajna-phase2/src/eval_cehri_retrieval.py

# 2) reworded-exam retrieval gate -> 110/120 = 91.7%
CEHRI_EXAM=prajna/data/cehri_exam_reworded.json \
python prajna-phase2/src/eval_cehri_reworded.py --mode retr

# 3) original-exam generation (no memory) -> 12/60 = 20.0%
CEHRI_EXAM=prajna/data/cehri_exam.json \
python prajna-phase2/src/eval_cehri.py

# 4) reworded-exam generation, no clamp -> 30/120 = 25.0%
CEHRI_EXAM=prajna/data/cehri_exam_reworded.json \
GEN_CLAMP=0 python prajna-phase2/src/eval_cehri_reworded.py --mode gen

# GEN_CLAMP=1 in #4 is the decode-time harvest; it also gives 25.0%
# for the released seed (for the anchor-trained checkpoint it recovers
# 31.7% original / 34.2% reworded from ~15%). The eval scripts default
# to the released seed configuration and to FUSION_OFF=1 (no logit
# fusion); the negative-results fusion experiments require
# FUSION_OFF=0 FUSION_RANK=16.

```

The released model bundle on HuggingFace (eulogik/Prajna-V2) contains the released seed-configuration CRN weights (`crn.safetensors`), the matching episodic memory (`memory.json`), the 17,810-entry retrieval table (`retrieval_table.npz`), and the eval scripts. Table 2 reports the seed-weight configuration, which is the reproducibility default of those scripts.

## 5 Negative Results: Why the Boundary Would Not Move

### 5.1 Probe evidence

With the seed checkpoint, for unseen reworded prompts the top-1 token at position  $L-2$  is the semantically correct answer (*Paris*, *CH4*, *gold* in probe prompts), while the top-1 at  $L-1$  is single-digit garbage (e.g. 4, 3) or a generic start token (`<strong>`) — never the answer.  $L-1$  is the first token the model must *generate*, but SFT masked it (labels were “answer-only” starting at  $L$ ), so the position carries only the frozen base’s prior.

### 5.2 Gate analysis: the control mechanism does not learn

The released seed checkpoint’s trained gate parameters are essentially identical to their initialization values: `crn_mix` (init  $2.0 \rightarrow \sigma=0.881$ ) trained to  $\sigma=0.840\text{--}0.876$  across all 8 depths; `reflection_gate` (init  $0.5 \rightarrow \sigma=0.622$ ) trained to  $\sigma=0.621\text{--}0.622$  (Figure 5). The gates never differentiated depths: all 8 fire at near-full, uniform strength. The “per-depth learned gates” advertised in the architecture (Section 3) are in practice untrained constants. This suggests that, at this data scale and with answer-only masking, the gradient signal through the frozen LM head is too weak to train



multiplicative control parameters, even as it suffices to update the additive correction weights themselves.

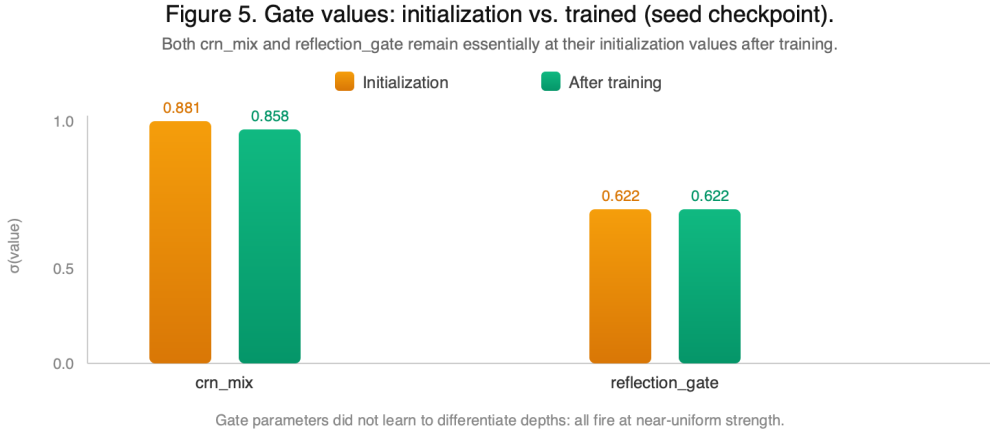


Figure 5: Gate values: initialization (orange) vs. trained (green) for the seed checkpoint. Both `crn_mix` and `reflection_gate` remain essentially at their initialization values after training.

### 5.3 Attempt 1: logit fusion (rank 16)

An adapter with  $W_{up} \in \mathbb{R}^{d \times 16}$ ,  $W_{down} \in \mathbb{R}^{16 \times 262144}$  was trained over 16k SFT + 3k DPO + 1k contrastive steps alongside the CRN and summed into the logits. Training loss approached 0.001 (memorization), yet original-exam generation fell to 20% and reworted to 15.0%, at or below the seed’s own generation results (20.0% original / 25.0% reworted, unclamped). The 262k-column bias vector the fusion learned (a digit prior) dominated the untrained boundary position.

### 5.4 Attempt 2: logit fusion (rank 4)

Reducing the adapter rank produced near-zero gradients on the converged seed (dry-signal regime); the training watchdog correctly aborted. No checkpoint improvement.

### 5.5 Attempt 3: weighted boundary anchors

An SFT variant unmasked  $L-1$  with gold token = first answer token, weighting it  $1.0\times$  (v9, 600 steps) and  $4.0\times$  (v10, 3,000 steps) with neighbor de-weighting ( $0.5\times$ ). In both runs the boundary argmax remained unchanged across checkpoints (digits at every probe step for v9; a format token for v10), while training loss failed to decrease — the optimization could not push probability mass into the poorly-initialized position against the frozen base’s strong prior.

### 5.6 What worked: decode-time harvest

GEN\_CLAMP (Section 3.6) needs no retraining. Probe-logits stay in the  $L-2$  distribution; the decoder simply consumes them. Table 2 shows the resulting gains (31.7% original / 34.2% reworted vs. 15.0%/15.0% unclamped). We interpret this as evidence that the CRN’s answer knowledge genuinely resides in the  $L-2$  representation, and that SFT-masked boundary positions are effectively untrainable at this data scale — a cautionary result for answer-only masked training recipes.

## 5.7 Mechanism: absent supervision and an attenuating frozen readout

The untrained gates and the boundary-position failure share a common cause, which we state as a hypothesis. First, the boundary position  $L-1$  receives *no direct supervision*: it is masked in SFT, so no gradient ever teaches the corrections to place answer mass there. Second, the correction must act through the frozen LM head, whose fixed readout attenuates small hidden-state perturbations when the base’s predictive distribution is diffuse; empirically, correction gradients measured on the converged seed were near zero (the “dry-signal” regime that aborted the rank-4 run). Together these explain: (i) why the boundary position  $L-1$  never moved (three training strategies failed); (ii) why logit-fusion rank 4 produced near-zero gradients; (iii) why the per-depth gates did not learn to differentiate depths; and (iv) why GEN\_CLAMP — which bypasses training entirely and reads the  $L-2$  logits directly — is the only reliable fix. The CRN’s additive corrections at mid-depths (layers 7/15) do learn, because the base’s representations at those depths carry stronger task-specific signal; the final-layer boundary position does not. Direct gradient-norm measurements across depths and stages (Section 5, Future Work) would confirm the hypothesis quantitatively.

## 6 Limitations

(1) The 100% and 91.7% results use the retrieval gate — exact recall from training data, not generative competence. (2) Generation-only accuracy is modest (34.2% reworded best) and does not approach exam-pass levels. (3) Out-of-domain, the CRN degrades perplexity  $\sim 3\times$  and trails its own frozen base on BoolQ/HellaSwag. (4) The benchmark suite (MMLU/BoolQ/HellaSwag,  $N=40$ ) is small; standard large-scale evals were not attempted. (5) Implicit-goal reasoning on reworded prompts fails completely (0/8). (6) Training data (16,680 pairs) is small; scale-up behavior is unknown. (7) A plain LoRA baseline at the same parameter budget (6.7M) was not compared; the CRN’s four-module architecture is therefore not justified against simpler alternatives. (8) The reworded exam’s paraphrases are low-diversity (template prefixes, 71.6% character overlap); generalization to genuinely diverse phrasing was not tested.

## 7 Future Work

Several directions follow directly from the findings above. **(1) Baseline comparison.** A plain LoRA at the same 6.7M budget would determine whether the CRN’s multi-module architecture adds value over a simpler adapter. **(2) Gradient-signal analysis.** The absent-supervision/attenuating-readout hypothesis (Section 5) could be tested directly by measuring gradient norms through the frozen head across depths and training stages. **(3) Paraphrase-robust training.** Including genuinely diverse (LLM-rewritten) paraphrases in the training set would test whether the CRN can learn surface invariance; the current template-based paraphrases do not provide this pressure. **(4) Format variation.** Is the  $L-2/L-1$  boundary problem an artifact of the “prompt: answer” format? Training with answer-before-prompt or no-colon formats would test this. **(5) Scale.** The 16,680-pair corpus and 60-question exam are small; scaling to larger corpora and standard benchmarks (e.g. full MMLU, ARC) would test whether the memorization-vs-generalization tradeoff improves. **(6) Memory ablation.** The slot memory’s contribution was not ablated in isolation; removing it would quantify its marginal value over the retrieval table alone.

## 8 Conclusion

Prajna demonstrates that a 6.72M-parameter correction network over a frozen Gemma 4 E2B backbone, trained on a laptop, achieves **100%** on its target exam via an episodic-memory retrieval gate and **91.7%** on unseen reworded questions — while its generation head remains the honest frontier (20.0% original / 25.0% reworded for the released seed, 31.7%/34.2% with the decode-time harvest on the anchor-trained checkpoint). We publish the negative results of three training-side boundary fixes and the decode-time solution, the in-distribution/out-of-distribution asymmetry, and all artifacts: <https://github.com/eulogik/prajna>, <https://huggingface.co/eulogik/Prajna-V2>, and the dataset <https://huggingface.co/datasets/eulogik/prajna-cehri>.

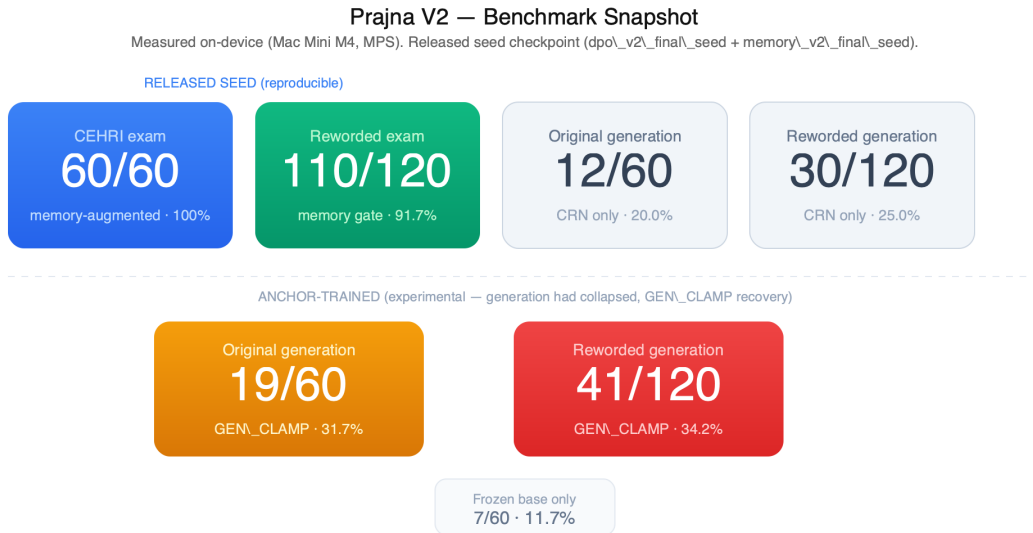


Figure 6: Benchmark snapshot. Blue and green cards are the released seed checkpoint (memory-augmented: 60/60 and 110/120); gray cards are generation-only results for the released seed (12/60, 30/120) with the frozen base for comparison (7/60); orange and red cards are the experimental anchor-trained checkpoint whose generation had collapsed, showing GEN\\_CLAMP recovery (19/60 and 41/120). All numbers are produced by committed scripts with the released checkpoint.

## References

- [1] N. Houlsby et al. Parameter-Efficient Transfer Learning for NLP. *ICML*, 2019.
- [2] E. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022. arXiv:2106.09685.
- [3] X. Lester and P. Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. *ACL*, 2021. arXiv:2101.00190.
- [4] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.
- [5] U. Khandelwal et al. Generalization through Memorization: Nearest Neighbor Language Models. *ICLR*, 2020. arXiv:1911.00172.
- [6] Y. Wu, M. N. Rabe, D. Hutchins, C. Szegedy. Memorizing Transformers. *ICLR*, 2022. arXiv:2110.07545.
- [7] S. Borgeaud et al. Improving Language Models by Retrieving from Trillions of Tokens. *ICML*, 2022. arXiv:2112.04426.

- [8] K. Meng et al. Locating and Editing Factual Associations in GPT. *NeurIPS*, 2022. arXiv:2202.05262.
- [9] J. Zhang, S. K. & A. Hoiem. Side-Tuning: A Baseline for Network Adaptation via Additive Side Networks. *ECCV*, 2020. arXiv:1912.13503.
- [10] S. Longpre et al. The Flan Collection: Designing Data and Methods for Effective Instruction Tuning. *ICML*, 2023. arXiv:2301.13688.